Name: Moheet Dialani
Roll no.: 2301028
Batch: T13

Experiment 3

LO4: Containerize, orchestrate, and deploy applications using Docker, Docker Compose, Kubernetes, and cloud infrastructure tools.

Aim: To deploy multi-service applications using Docker Compose.

Theory:

Docker Compose is a tool used to define and manage multi-container Docker applications. Instead of running multiple docker run commands for each container, Docker Compose allows all services to be configured in a single **docker-compose.yml** file. This file specifies the services, images, ports, networks, volumes, and environment variables required for the application.

A multi-service application consists of two or more containers that work together. For example, a web application may require a web server, a database, and a caching service. Docker Compose automatically creates a network that enables these containers to communicate with each other using their service names.

To deploy an application, Docker Compose reads the docker-compose.yml file, builds any required images, creates the necessary containers, configures networking and storage, and starts all services with a single command:

docker compose up --build

The --build option ensures that the Docker images are rebuilt before the containers are started. Docker Compose also manages the lifecycle of the application, allowing users to stop and remove all services with:

docker compose down

Advantages of Docker Compose

- Simplifies deployment of multi-container applications.
- Uses a single configuration file for all services.
- Automatically creates a network for container communication.

Name: Moheet Dialani
Roll no.: 2301028
Batch: T13

- Supports persistent storage using volumes.
- Makes development, testing, and deployment consistent across different environments.
- Reduces manual configuration and improves maintainability.

Applications

- Web applications with backend and database services.
- Microservices-based applications.
- Development and testing environments.
- Continuous Integration and Continuous Deployment (CI/CD) workflows.

Commands:

Command	**Purpose**
docker compose up	Start all services.
docker compose up --build	Build images and start services.
docker compose up -d	Run services in the background.
docker compose ps	View running containers.
docker compose logs	View container logs.
docker compose stop	Stop services without removing them.
docker compose restart	Restart services.
docker compose down	Stop and remove all services, containers, and networks.

Name: Moheet Dialani
Roll no.: 2301028
Batch: T13

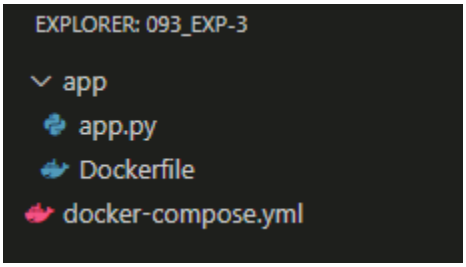
Implementation:

```
C:\Users\15L>docker compose version
Docker Compose version v2.24.6-desktop.1

C:\Users\15L>docker --version
Docker version 25.0.3, build 4debf41
```

Step 1: Create the Project Directory

File structure:



Step 2: create docker file

Dockerfile:

```
app > Dockerfile > ...
1   FROM python:3.11
2
3   WORKDIR /app
4
5   COPY . .
6
7   RUN pip install -r requirements.txt
8
9   CMD ["python", "app.py"]
```

Step 3:create docker-compose.yml

Docker-compose.yml:

Name: Moheet Dialani
Roll no.: 2301028
Batch: T13

docker-compose.yml

```yaml
version: "3.9"

services:
  web:
    build: ./app
    ports:
      - "5000:5000"
  db:
    image: mysql:8.0
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: testdb
    ports:
      - "3306:3306"
```

App.py:

app > app.py > ...

```python
from flask import Flask
import mysql.connector

app = Flask(__name__)

@app.route("/")
def home():
    return "Flask is running with MySQL using Docker Compose!"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Step 4:build and start the services

Name: Moheet Dialani
Roll no.: 2301028
Batch: T13

```
C:\Users\15L>cd desktop

C:\Users\15L\Desktop>cd 093_exp-3

C:\Users\15L\Desktop\093_exp-3>docker compose up --build
[+] Building 0.0s (0/0)  docker:default
2026/07/29 14:01:43 http2: server: error reading preface from client //./pipe/docker_engine: file has already been close[+] Building 1.7s (9/9) FINISHED
           docker:default
 => [web internal] load build definition from Dockerfile                                                0.0s
 => => transferring dockerfile: 134B                                                                    0.0s
 => [web internal] load metadata for docker.io/library/python:3.11                                      1.6s
 => [web internal] load .dockerignore                                                                   0.0s
 => => transferring context: 2B                                                                         0.0s
 => [web 1/4] FROM docker.io/library/python:3.11@sha256:c7228863385ee39fb6d822da81f4a69d8cd33ff893d92ce94185e5c3f  0.0s
 => [web internal] load build context                                                                   0.0s
 => => transferring context: 2.89B                                                                      0.0s
 => CACHED [web 2/4] WORKDIR /app                                                                       0.0s
 => CACHED [web 3/4] COPY . .                                                                           0.0s
 => CACHED [web 4/4] RUN pip install flask                                                              0.0s
 => [web] exporting to image                                                                            0.0s
 => => exporting layers                                                                                 0.0s
 => => writing image sha256:ea09207fa7a766d27b6c7ea0581b6ece8054562a1c7251ba600c588cc5d2b6c4            0.0s
 => => naming to docker.io/library/093_exp-3-web                                                        0.0s
[+] Running 1/0
 ✔Container 093_exp-3-web-1  Recreated                                                                  0.1s
Attaching to web-1
web-1  |  * Serving Flask app 'app'
web-1  |  * Debug mode: off
web-1  | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
web-1  |  * Running on all addresses (0.0.0.0)
web-1  |  * Running on http://127.0.0.1:5000
web-1  |  * Running on http://172.21.0.2:5000
web-1  | Press CTRL+C to quit
```

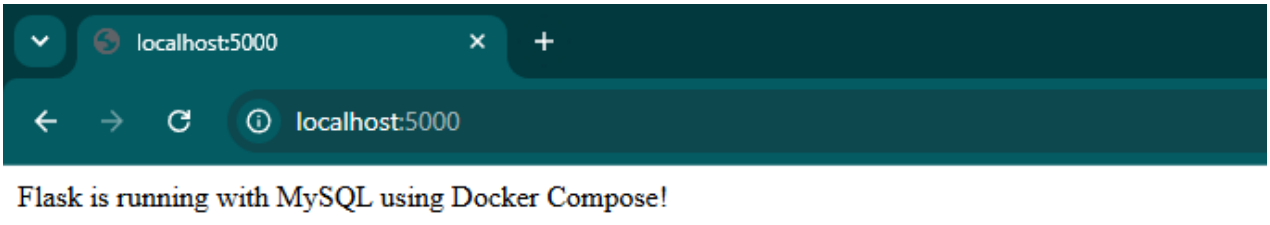
Step 5: verify the deployment

```
PS C:\Users\15L> cd desktop
PS C:\Users\15L\desktop> cd 093_exp-3
PS C:\Users\15L\desktop\093_exp-3> docker compose ps
NAME              IMAGE             COMMAND           SERVICE   CREATED         STATUS         PORTS
093_exp-3-web-1   093_exp-3-web     "python app.py"   web       2 minutes ago   Up 2 minutes   0.0.0.0:5000->5000/tcp
```

Step 6: view application logs

```
PS C:\Users\15L\desktop\093_exp-3> docker compose logs
web-1  |  * Serving Flask app 'app'
web-1  |  * Debug mode: off
web-1  | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server i
nstead.
web-1  |  * Running on all addresses (0.0.0.0)
web-1  |  * Running on http://127.0.0.1:5000
web-1  |  * Running on http://172.21.0.2:5000
web-1  | Press CTRL+C to quit
```

Step 7: open project on local host:5000



Step 8: stop and remove services

Name: Moheet Dialani
Roll no.: 2301028
Batch: T13

```
PS C:\Users\15L\desktop\093_exp-3> docker compose down
[+] Running 2/2
 ✔Container 093_exp-3-web-1  Removed                                    10.3s
 ✔Network 093_exp-3_default  Removed                                     0.2s
PS C:\Users\15L\desktop\093_exp-3>
```

Conclusion:

Docker Compose is an efficient tool for deploying and managing multi-service applications using a single configuration file. It automates the process of building images, creating containers, configuring networks, and starting services with minimal effort. By simplifying container orchestration, Docker Compose ensures consistency across development, testing, and deployment environments, making application management easier, faster, and more reliable.